

PRACTICAL 2

Program1:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    // 1. Open an existing file for reading
    int fd_read = open("source.txt", O_RDONLY);
    if (fd_read < 0) {
        perror("Error opening source file");
    } else {
        printf("Successfully opened 'source.txt' with File Descriptor: %d\n", fd_read);
        close(fd_read);
    }

    // 2. Open/Create a new file for writing with permissions (0644)
    int fd_write = open("destination.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd_write < 0) {
        perror("Error creating destination file");
    } else {
        printf("Successfully opened/created 'destination.txt' with File Descriptor: %d\n", fd_write);
        close(fd_write);
    }

    return 0;
}
```

Output:

```
nishanth@LAPTOP-6Q30N07H:~/practical2$ ./program1
Error opening source file: No such file or directory
Successfully opened/created 'destination.txt' with File Descriptor: 3
```

Program 2:

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 128

int main() {
    int fd = open("source.txt", O_RDONLY);
    if (fd < 0) {
        perror("Error opening file to read");
        exit(EXIT_FAILURE);
    }

    char buffer[BUFFER_SIZE];

    // Read up to BUFFER_SIZE bytes from file descriptor into memory buffer
    ssize_t bytes_read = read(fd, buffer, sizeof(buffer) - 1);
    if (bytes_read < 0) {
        perror("Error reading file");
    } else {
        buffer[bytes_read] = '\0'; // Null-terminate string
        printf("Read %ld bytes from file:\n%s\n", bytes_read, buffer);
    }

    close(fd);
    return 0;
}

```

Output:

```

nishanth@LAPTOP-6Q3ON07H:~/practical2$ ./program2
Read 29 bytes from file:
Hello, this is test content.

```

Program3:

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd = open("destination.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("Error opening file to write");
        exit(EXIT_FAILURE);
    }

    char *text = "Hello! This text is written directly using the write() system call.\n";

    // Write buffer contents to file descriptor
    ssize_t bytes_written = write(fd, text, strlen(text));
    if (bytes_written < 0) {
        perror("Error writing to file");
    } else {
        printf("Successfully wrote %ld bytes to 'destination.txt'\n", bytes_written);
    }

    close(fd);
    return 0;
}

```

Output:

```

nishanth@LAPTOP-6Q3ON07H:~/practical2$ ./program3
Successfully wrote 68 bytes to 'destination.txt'

```

Program 4:

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("source.txt", O_RDONLY);
    if (fd < 0) {
        perror("Failed to open file");
        exit(EXIT_FAILURE);
    }

    printf("File opened successfully. File Descriptor ID: %d\n", fd);

    // Explicitly closing the file descriptor release kernel resources
    if (close(fd) == 0) {
        printf("File descriptor %d closed successfully.\n", fd);
    } else {
        perror("Error closing file descriptor");
    }

    return 0;
}

```

Output:

```

nishanth@LAPTOP-6Q3ON07H:~/practical2$ ./program4
File opened successfully. File Descriptor ID: 3
File descriptor 3 closed successfully.

```

History :

```
nishanth@LAPTOP-6Q3ON07H:~/practical2$ history
```

```
1  sudo apt update
2  sudo apt install build-essential
3  gcc --version
4  mkdir OSLab
5  cd OSLab
6  gedit hello.c
7  sudo apt install gedit
8  gedit hello.c
9  sudo apt update
10 sudo apt install gcc
11 gcc --version
12 ls
13 gedit hello.c
14 mkdir OSLab
15 cd OSLab
16 gedit hello.c
17 nano process.c
18 ls
19 gcc process.c -o process
20 ./process
21 uname
22 lscpu
23 lsblk
24 ps
25 top
26 echo $shell
27 echo $SHELL
28 nano demo.c
29 gcc demo.c -o demo
30 ./demo
31 help
32 uname -a
33 mkdir fork
34 cd fork
35 nano program1.c
36 gcc program1.c -o program1
37 nano program1.c
38 gcc program1.c -o program1
39 ./program1
40 nano program2
41 gcc program1.c -o program2
42 ./program2
43 ./program1
44 nano program1.c
45 ls
46 rm program2
47 ls
48 nano program2.c
49 gcc program2.c -o program2
```

```
65 ./program4
66 nano program7.c
67 gcc program7.c -o program7
68 ./program7
69 nano program8.c
70 gcc program8.c -o program8
71 ./program8
72 cd ~/fork
73 ls
74 cat program1.c
75 ./program1
76 cat program2.c
77 ./program2
78 cat progra3.c
79 cat program3.c
80 ./program3
81 cat program4.c
82 ./program4
83 cat program5.c
84 ./program5
85 cat program6.c
86 ./program6
87 cat program7.c
88 ./program7
89 cat program8.c
90 ./program8
91 history
92 history | sed -n '33,71p'
93 nano program9.c
94 gcc program9.c -o program9
95 ./program9
96 history
97 cd ~/fork
98 ls
99 cat program1.c
100 ./program1
101 pwd
102 cd OSLAB
103 mkdir OSLAB
104 cd OSLAB
105 mkdir exec
106 cd exec
107 nano program1.c
108 gcc program1.c -o progarm1
109 ./program1
110 gcc program1.c -o program1
111 ./progarm1.c
112 ./program1
113 nano program2.c
114 gcc program2.c -o program2
```

```
128 ./program2
129 cat program3.c
130 ./program3
131 cat program4.c
132 ./program4
133 cat program5.c
134 ./program5
135 mkdir practical1
136 cd practical1
137 nano program1.c
138 gcc program1.c -o program1
139 ./program1.c
140 ./program1
141 cat program1.c
142 nano program2.c
143 gcc program2.c -o program2
144 ./program2
145 nano program3.c
146 gcc program3.c -o program3
147 ./program3
148 nano program4.c
149 gcc program4.c -o program4
150 ./program4
151 nano program5.c
152 gcc program5.c -o program5
153 ./program5
154 uname -a
155 lscpu
156 lsblk
157 ps
158 top
159 ls
160 history
161 mkdir practical2
162 cd practical2
163 pwd
164 mkdir practical2
165 cd practical2
166 nano program1.c
167 gcc program1.c -o program1
168 ./program2
169 ./program1
170 nano program2.c
171 gcc program2.c -o program2
172 ./program2
173 echo "Hello, this is test content." > source.txt
174 ls
175 ./program2
176 nano program3.c
177 gcc program3.c -o program3
```

